

[API TESTING GUIDE]

API TESTING

Complete Guide: Basic to Advanced

Everything a QA Engineer needs to know — from HTTP basics to full automation with Python, Postman & CI/CD pipelines.

■ HTTP Methods

■ Request Anatomy

■ Status Codes

■ Assertions

■ Auth Testing

■ Performance

■ Tools

■ Automation

■ CI/CD

25+ Pages | Real Code Examples | Interview Ready | LinkedIn Carousel

13 Topics

25+ Code
Snippets

50+ Test Cases

28 Pages

LEARNING ROADMAP

BASIC HTTP Methods Status
Codes Request Anatomy
Postman Manual



INTERMEDIATE Python
Requests Pytest Framework
Assertions Auth Testing



ADVANCED Negative
Testing Performance CI/CD
Pipeline Load Testing

Table of Contents

Your complete API testing roadmap

01	What is API Testing?	Basics, why it matters, types
02	HTTP Methods	GET, POST, PUT, PATCH, DELETE
03	Anatomy of a Request	URL, Headers, Body, Auth
04	HTTP Status Codes	2xx, 3xx, 4xx, 5xx explained
05	Writing Assertions	What to validate, how to assert
06	Postman — Manual Testing	Collections, environments, tests
07	Python + Requests Library	Script-based API testing
08	Pytest for API Automation	Framework, fixtures, reports
09	Authentication Testing	Bearer, Basic, OAuth2, API Keys
10	Negative & Edge Case Testing	Error flows, boundary tests
11	Performance & Load Testing	Response time, locust basics
12	CI/CD Integration	Jenkins, GitHub Actions, Newman
13	Interview Cheat Sheet	Top Q&A; for your next interview

What is API Testing?

The backbone of modern software quality

API (Application Programming Interface) Testing is the process of validating that your API endpoints work correctly — checking requests, responses, data integrity, authentication, and performance WITHOUT going through the UI.

- API testing finds 80% of backend bugs before they ever reach the frontend. It's 5x faster than UI testing and runs in every CI/CD pipeline.

Why API Testing?

- Faster feedback — no browser rendering needed
- More stable — APIs don't change layout like UIs do
- Covers business logic directly
- Runs in headless CI/CD environments easily
- Catches data contract violations early

Types of API Testing

Type	What it checks	Example
Functional	API does what it's supposed to	POST /login returns 200 with token
Negative	API handles invalid input	POST /login with wrong password → 401
Performance	Response time under load	GET /products responds < 500ms
Security	Auth, injection, data exposure	Access without token → 401
Contract	Schema/structure is correct	Response has required fields

HTTP Methods

The 5 verbs you MUST know cold

HTTP Methods tell the server WHAT action to perform on a resource. Each method has a clear semantic meaning and expected behavior.

Method	Purpose	Request Body	Success Code
GET	Retrieve/Read a resource	■ None	200 OK
POST	Create a new resource	■ Required	201 Created
PUT	Replace entire resource	■ Required	200 OK
PATCH	Partially update a resource	■ Required	200 OK
DELETE	Delete a resource	■ Optional	204 No Content

Key Differences to Remember

■ PUT replaces the ENTIRE resource. PATCH updates only specified fields. This is a common interview trap!

■ GET and DELETE have no request body. POST/PUT/PATCH send data in the body.

Idempotency — Interview Gold ■

An operation is **idempotent** if calling it multiple times produces the same result:

Method	Idempotent ?	Safe?	Why
GET	■ Yes	■ Yes	Same response every time, no side effects
POST	■ No	■ No	Creates new resource each call
PUT	■ Yes	■ No	Replaces same resource every time
PATCH	■ No	■ No	May have cumulative effects
DELETE	■ Yes	■ No	First deletes, subsequent return 404

Anatomy of a Request

Every component you must understand & test

Endpoint URL	The full address of the resource: <code>https://api.example.com/v1/users/{id}</code>
HTTP Method	The action verb – GET, POST, PUT, PATCH, DELETE
Headers	Metadata sent with the request
Request Body	Data payload (POST/PUT/PATCH only)
Query Params	Filter/search params in the URL

Complete Request Example

request_anatomy.py

```
1 # Complete API Request anatomy in Python
2 import requests
3
4 url = 'https://api.example.com/v1/users'
5
6 # Headers
7 headers = {
8     'Content-Type': 'application/json',
9     'Authorization': 'Bearer eyJhbGciOiJIUzI1NiJ9...',
10    'Accept': 'application/json',
11    'X-API-Version': '2024-01'
12 }
13
14 # Request Body (for POST)
15 payload = {
16     'name': 'Abhijeet Malviya',
17     'email': 'abhijeet@test.com',
18     'role': 'qa_engineer'
19 }
20
21 # Query Parameters
22 params = {'active': True, 'page': 1}
23
24 # Send Request
```

```
25 response = requests.post(url, json=payload, headers=headers, params=params)
26 print(response.status_code) # 201
27 print(response.json())      # {'id': 'usr_123', ...}
```

HTTP Status Codes

Know every code — interviewers love these!

HTTP Status Codes tell you the result of your request. A QA engineer must instantly know what each code means and when it should appear.

	Code	Meaning	When it happens
2xx — SUCCESS	200	OK	Successful GET, PUT, PATCH
	201	Created	Successful POST — resource created
	202	Accepted	Request accepted, processing async
	204	No Content	Successful DELETE — no body returned
	Code	Meaning	When it happens
3xx — REDIRECT	301	Moved Permanently	Resource URL changed forever
	302	Found	Temporary redirect
	304	Not Modified	Cached version is still valid
	Code	Meaning	When it happens
4xx — CLIENT ERROR	400	Bad Request	Invalid request body or params
	401	Unauthorized	Missing or invalid authentication
	403	Forbidden	Authenticated but no permission
	404	Not Found	Resource doesn't exist
	405	Method Not Allowed	Wrong HTTP method used
	409	Conflict	Duplicate resource / state conflict
	422	Unprocessable	Validation failed on request data
	429	Too Many Requests	Rate limit exceeded

	Code	Meaning	When it happens
5xx — SERVER ERROR	500	Internal Server Error	Unexpected server crash
	502	Bad Gateway	Upstream server error
	503	Service Unavailable	Server is down / overloaded
	504	Gateway Timeout	Upstream server timed out

401 vs 403: 401 means 'I don't know who you are'. 403 means 'I know who you are, but you can't do this'. This distinction is a classic interview question!

Writing API Assertions

The heart of any good API test

An assertion validates that the API response matches the expected outcome. Every API test should assert multiple things — not just the status code.

The 6 Things You Must Always Assert

- **Status Code** Is the response code correct for this operation?
- **Response Time** Did it respond within acceptable time? (< 2s typical)
- **Schema/Fields** Are all required fields present in the response?
- **Data Types** Are field types correct? (string, int, bool, array?)
- **Content-Type** Is Content-Type: application/json in response headers?
- **Auth Behavior** Without token → 401? Wrong token → 401?

Python Assertion Examples

test_assertions.py

```
1 import requests
2 import pytest
3
4 BASE_URL = 'https://api.example.com/v1'
5
6 def test_create_user():
7     payload = {'name': 'Abhijeet', 'email': 'ab@test.com', 'role': 'qa'}
8     headers = {'Authorization': 'Bearer valid_token_here'}
9
10    response = requests.post(f'{BASE_URL}/users', json=payload, headers=headers)
11
12    # 1. Status Code
13    assert response.status_code == 201, f'Expected 201, got {response.status_code}'
14
15    # 2. Response Time
16    assert response.elapsed.total_seconds() < 2.0, 'Response too slow!'
17
```

```
18 data = response.json()
19
20 # 3. Schema – required fields
21 assert 'id' in data, 'Missing field: id'
22 assert 'name' in data, 'Missing field: name'
23 assert 'email' in data, 'Missing field: email'
24
25 # 4. Data Types
26 assert isinstance(data['id'], str), 'id must be string'
27 assert isinstance(data['name'], str), 'name must be string'
28
29 # 5. Content-Type Header
30 assert 'application/json' in response.headers['Content-Type']
31
32 # 6. Data Integrity
33 assert data['email'] == 'ab@test.com', 'Email mismatch!'
34 assert data['name'] == 'Abhijeet', 'Name mismatch!'
```

Postman — Manual API Testing

The most popular API testing tool

Postman is a GUI tool for sending API requests, organizing test collections, managing environments, and running automated test scripts.

Postman Key Concepts

Collection	Group of organized API requests — like a test suite
Environment	Variables for different configs (dev, staging, prod)
Variables	{{base_url}}, {{token}} — reusable dynamic values
Pre-request Script	JavaScript that runs BEFORE the request
Tests Script	JavaScript that runs AFTER — your assertions live here

Postman Test Script (JavaScript)

postman_tests.js

```
1 // Postman Tests Tab — JavaScript
2
3 // 1. Status Code
4 pm.test('Status is 201', () => {
5     pm.response.to.have.status(201);
6 });
7
8 // 2. Response Time
9 pm.test('Response time < 2000ms', () => {
10     pm.expect(pm.response.responseTime).to.be.below(2000);
11 });
12
13 // 3. JSON field exists
14 pm.test('Has user id', () => {
15     const data = pm.response.json();
16     pm.expect(data).to.have.property('id');
17 });
18
19 // 4. Save token for next request
20 const token = pm.response.json().token;
21 pm.environment.set('auth_token', token);
```

```
22
23 // 5. Content-Type header
24 pm.test('Content-Type is JSON', () => {
25     pm.response.to.have.header('Content-Type', /json/);
26 });
```



Run your Postman collection from terminal: `newman run collection.json -e env.json --reporters html`

Python — requests Library

Script-based API testing for automation

The Python requests library is the go-to for API automation. It's simple, powerful, and integrates perfectly with Pytest.

installation

```
1 pip install requests
2 pip install pytest
3 pip install pytest-html # for reports
```

All HTTP Methods in Python

http_methods.py

```
1 import requests
2
3 BASE = 'https://api.example.com/v1'
4 HEADERS = {'Authorization': 'Bearer your_token'}
5
6 # GET – Read
7 r = requests.get(f'{BASE}/users', headers=HEADERS)
8 r = requests.get(f'{BASE}/users/123', headers=HEADERS) # single user
9 r = requests.get(f'{BASE}/users', params={'role': 'admin'}, headers=HEADERS)
10
11 # POST – Create
12 r = requests.post(f'{BASE}/users',
13                  json={'name': 'Abhijeet', 'email': 'ab@test.com'},
14                  headers=HEADERS)
15
16 # PUT – Full Update
17 r = requests.put(f'{BASE}/users/123',
18                  json={'name': 'Abhijeet M', 'email': 'new@test.com'},
19                  headers=HEADERS)
20
21 # PATCH – Partial Update
22 r = requests.patch(f'{BASE}/users/123',
23                   json={'role': 'senior_qa'},
24                   headers=HEADERS)
25
26 # DELETE
27 r = requests.delete(f'{BASE}/users/123', headers=HEADERS)
28
29 # Response inspection
```

```
30 print(r.status_code)           # 200
31 print(r.json())                # dict
32 print(r.headers)               # response headers
33 print(r.elapsed.total_seconds()) # response time
```

Session Management

session.py

```
1  # Use Session for persistent auth & performance
2  session = requests.Session()
3  session.headers.update({
4      'Authorization': 'Bearer your_token',
5      'Content-Type': 'application/json'
6  })
7
8  # All requests use session headers automatically
9  r1 = session.get('https://api.example.com/v1/users')
10 r2 = session.post('https://api.example.com/v1/users', json={...})
11 session.close()
```

Pytest for API Automation

Building a real test suite

Project Structure

project_structure

```
1  api_tests/
2  ███ conftest.py          # Shared fixtures
3  ███ pytest.ini          # Pytest config
4  ███ tests/
5  █   ███ test_users.py
6  █   ███ test_auth.py
7  █   ███ test_products.py
8  ███ utils/
9      ███ api_client.py    # Reusable request wrapper
10     ███ helpers.py
```

conftest.py — Fixtures

conftest.py

```
1  # conftest.py – shared setup for all tests
2  import pytest
3  import requests
4
5  BASE_URL = 'https://api.example.com/v1'
6
7  @pytest.fixture(scope='session')
8  def auth_token():
9      '''Login once, reuse token across all tests'''
10     resp = requests.post(f'{BASE_URL}/auth/login',
11                          json={'email': 'test@example.com', 'password': 'Test@123'})
12     assert resp.status_code == 200
13     return resp.json()['token']
14
15  @pytest.fixture
16  def api_client(auth_token):
17      '''Preconfigured session with auth headers'''
18     session = requests.Session()
19     session.headers.update({
20         'Authorization': f'Bearer {auth_token}',
21         'Content-Type': 'application/json'
22     })
```

```

23     yield session
24     session.close()
25
26 @pytest.fixture
27 def create_user(api_client):
28     '''Create a user and cleanup after test'''
29     resp = api_client.post('/users',
30         json={'name': 'Test User', 'email': 'test@mail.com'})
31     user_id = resp.json()['id']
32     yield resp.json()          # Test uses this
33     api_client.delete(f'/users/{user_id}') # Cleanup

```

test_users.py — Full Test File

test_users.py

```

1  import pytest
2  BASE = 'https://api.example.com/v1'
3
4  class TestUsersAPI:
5      def test_get_all_users(self, api_client):
6          r = api_client.get(f'{BASE}/users')
7          assert r.status_code == 200
8          assert isinstance(r.json(), list)
9          assert r.elapsed.total_seconds() < 2
10
11     def test_create_user_success(self, api_client):
12         payload = {'name': 'Abhijeet', 'email': 'ab@test.com'}
13         r = api_client.post(f'{BASE}/users', json=payload)
14         assert r.status_code == 201
15         data = r.json()
16         assert 'id' in data
17         assert data['email'] == 'ab@test.com'
18
19     @pytest.mark.parametrize('email', ['', 'notanemail', 'a@', '@b.com'])
20     def test_create_user_invalid_email(self, api_client, email):
21         r = api_client.post(f'{BASE}/users', json={'email': email})
22         assert r.status_code == 422
23
24     def test_update_user(self, api_client, create_user):
25         user_id = create_user['id']
26         r = api_client.patch(f'{BASE}/users/{user_id}', json={'role': 'admin'})
27         assert r.status_code == 200
28         assert r.json()['role'] == 'admin'
29
30     def test_delete_user(self, api_client, create_user):

```



```
31     user_id = create_user['id']
32     r = api_client.delete(f'{BASE}/users/{user_id}')
33     assert r.status_code == 204
34     # Verify it's gone
35     r2 = api_client.get(f'{BASE}/users/{user_id}')
36     assert r2.status_code == 404
```

Run Tests

terminal commands

```
1  # Run all tests
2  pytest tests/ -v
3
4  # Run with HTML report
5  pytest tests/ -v --html=report.html
6
7  # Run specific test
8  pytest tests/test_users.py::TestUsersAPI::test_create_user_success -v
9
10 # Run by marker
11 pytest tests/ -m 'smoke' -v
```

Authentication Testing

Security layer — never skip this!

Authentication testing verifies that your API correctly grants and denies access. A QA engineer must test every auth scenario.

Types of Authentication

Bearer Token (JWT)

- Most common in REST APIs
- Sent in Authorization header
- Authorization: Bearer eyJhbGciOiJIUzI1...
- Test: valid token → 200, no token → 401, expired token → 401

API Key

- Simple key sent in header or query param
- X-API-Key: your_api_key_here
- Or: GET /users?api_key=your_key
- Test: valid key → 200, invalid key → 401 or 403

Basic Auth

- Username:Password encoded as Base64
- Authorization: Basic dXNlcjpwYXNz
- Used in legacy systems
- Test: wrong credentials → 401

OAuth2

- Token obtained via flow (client_credentials, authorization_code)
- Access token + Refresh token
- Test: expired access token, use refresh token
- Scopes control permissions

Auth Test Scenarios in Python

test_auth.py

```
1 import requests, pytest
2 BASE = 'https://api.example.com/v1'
3
4 class TestAuthentication:
5     def test_no_token_returns_401(self):
6         r = requests.get(f'{BASE}/users') # No auth header
7         assert r.status_code == 401
8
9     def test_invalid_token_returns_401(self):
```

```
10     r = requests.get(f'{BASE}/users',
11                       headers={'Authorization': 'Bearer invalid_token'})
12     assert r.status_code == 401
13
14 def test_expired_token_returns_401(self):
15     expired = 'eyJhbGciOiJIUzI1NiJ9.eyJleHAiOjF9.fake'
16     r = requests.get(f'{BASE}/users',
17                       headers={'Authorization': f'Bearer {expired}'})
18     assert r.status_code == 401
19
20 def test_valid_token_returns_200(self, auth_token):
21     r = requests.get(f'{BASE}/users',
22                       headers={'Authorization': f'Bearer {auth_token}'})
23     assert r.status_code == 200
24
25 def test_wrong_role_returns_403(self, viewer_token):
26     # viewer_token has read-only permissions
27     r = requests.delete(f'{BASE}/users/123',
28                         headers={'Authorization': f'Bearer {viewer_token}'})
29     assert r.status_code == 403 # Forbidden (not 401!)
```

Negative & Edge Case Testing

Where most QAs stop — where great QAs shine

Negative testing verifies that your API handles invalid inputs gracefully, returns proper error codes, and doesn't expose sensitive data in errors.

Negative Test Scenarios

Scenario	Input	Expected
Missing Required Fields	POST /users with no email	422 Unprocessable
Wrong Data Type	age: 'twenty' instead of 20	422 Unprocessable
Duplicate Resource	POST /users with existing email	409 Conflict
Non-existent Resource	GET /users/99999	404 Not Found
Invalid Auth	Bearer invalid_token_xyz	401 Unauthorized
Forbidden Action	DELETE with read-only role	403 Forbidden
Oversized Payload	Request body > 10MB	413 Too Large
SQL Injection Attempt	name: '; DROP TABLE users;--'	400 or sanitized
Empty String Fields	name: '' (empty string)	422 Unprocessable
Boundary Values	age: -1 or age: 999	422 Unprocessable

Parametrized Negative Tests

```
test_negative.py
1  import pytest, requests
2  BASE = 'https://api.example.com/v1'
3
4  @pytest.mark.parametrize('payload,expected_code', [
5      # Missing fields
6      ({}, 422), # No fields
7      ({'name': 'Test'}, 422), # Missing email
8      ({'email': 'test@mail.com'}, 422), # Missing name
9      # Invalid formats
10     ({'name': '', 'email': 'test@mail.com'}, 422), # Empty name
11     ({'name': 'Test', 'email': 'notanemail'}, 422), # Bad email
```

```
12     ({'name': 'A', 'email': 'a@b.c'}, 422), # Name too short
13     # Duplicates handled separately
14 ])
15 def test_create_user_invalid_inputs(payload, expected_code, api_client):
16     r = api_client.post(f'{BASE}/users', json=payload)
17     assert r.status_code == expected_code, (
18         f'Payload: {payload} → Expected {expected_code}, got {r.status_code}'
19     )
20     # Also verify error message exists
21     assert 'error' in r.json() or 'message' in r.json()
```

Performance & Load Testing

APIs must be fast AND stable under load

Response Time Benchmarks

Category	Response Time	User Experience
Excellent	< 200ms	Feels instant
Good	200-500ms	Acceptable for most APIs
Acceptable	500ms-1s	User might notice
Slow	1s-2s	Degraded UX, investigate
Critical	> 2s	Test FAILS — must fix

Response Time Assertions in Python

```
test_performance.py
1  import requests, time
2
3  def test_api_performance():
4      url = 'https://api.example.com/v1/products'
5
6      # Single request timing
7      r = requests.get(url)
8      assert r.elapsed.total_seconds() < 0.5, f'Too slow: {r.elapsed.total_seconds()}s'
9
10     # Multiple requests - average timing
11     times = []
12     for _ in range(10):
13         start = time.time()
14         requests.get(url)
15         times.append(time.time() - start)
16
17     avg = sum(times) / len(times)
18     p95 = sorted(times)[int(len(times) * 0.95)]
19     print(f'Avg: {avg:.3f}s | P95: {p95:.3f}s')
20     assert avg < 0.5, f'Avg response too slow: {avg:.3f}s'
21     assert p95 < 1.0, f'P95 too slow: {p95:.3f}s'
```

Basic Load Test with Locust

locustfile.py

```
1  # pip install locust
2  # locustfile.py
3  from locust import HttpUser, task, between
4
5  class APIUser(HttpUser):
6      wait_time = between(1, 2) # Wait 1-2s between tasks
7
8      def on_start(self):
9          '''Login once per user'''
10         resp = self.client.post('/auth/login',
11                                 json={'email': 'test@mail.com', 'password': 'Test@123'})
12         self.token = resp.json()['token']
13         self.client.headers['Authorization'] = f'Bearer {self.token}'
14
15     @task(3) # 3x more frequent than weight 1
16     def get_users(self):
17         self.client.get('/v1/users')
18
19     @task(1)
20     def create_user(self):
21         self.client.post('/v1/users',
22                         json={'name': 'Load Test', 'email': f'lt@test.com'})
23
24     # Run: locust --headless -u 50 -r 5 --host=https://api.example.com
```

CI/CD Integration

Automate your tests in every pipeline

API tests should run automatically on every code push. This catches regressions before they reach production.

GitHub Actions Workflow

.github/workflows/api-tests.yml

```
1  # .github/workflows/api-tests.yml
2  name: API Test Suite
3
4  on:
5    push:
6      branches: [main, develop]
7    pull_request:
8      branches: [main]
9
10 jobs:
11   api-tests:
12     runs-on: ubuntu-latest
13     steps:
14       - uses: actions/checkout@v3
15
16       - name: Set up Python
17         uses: actions/setup-python@v4
18         with:
19           python-version: '3.11'
20
21       - name: Install dependencies
22         run: pip install -r requirements.txt
23
24       - name: Run API Tests
25         env:
26           API_BASE_URL: ${ secrets.API_BASE_URL }
27           API_TOKEN: ${ secrets.API_TOKEN }
28         run: pytest tests/ -v --html=report.html --tb=short
29
30       - name: Upload Report
31         uses: actions/upload-artifact@v3
32         if: always()
33         with:
```



```
34     name: test-report
35     path: report.html
```

Newman (Postman CLI) in CI/CD

newman_ci.sh

```
1  # Install Newman
2  npm install -g newman newman-reporter-html
3
4  # Run Postman collection in CI
5  newman run collection.json \
6    --environment staging.json \
7    --reporters cli,html \
8    --reporter-html-export newman-report.html \
9    --bail # Stop on first failure
10
11 # In GitHub Actions:
12 # - name: Run Newman Tests
13 #   run: |
14 #     newman run tests/api-collection.json \
15 #       -e tests/env-staging.json \
16 #       --reporters cli,html
```

■ Best Practice: Run smoke tests on every push, full test suite nightly. Tag tests with @smoke, @regression, @slow markers.

Interview Cheat Sheet

Top questions + answers for your next QA interview

Q: Difference between 401 and 403?

401 = Not authenticated (no/invalid token). 403 = Authenticated but no permission to do this action.

Q: What is idempotency?

An operation is idempotent if repeating it gives the same result. GET, PUT, DELETE are idempotent. POST and PATCH are not.

Q: What do you test in an API response?

Status code, response time, JSON schema/fields, data types, data integrity, Content-Type header, error message structure.

Q: Difference between PUT and PATCH?

PUT replaces the ENTIRE resource. PATCH updates only the fields you send. PUT is idempotent, PATCH may not be.

Q: How do you handle auth in Pytest API tests?

Use a session-scoped fixture in conftest.py that logs in once and returns the token. Inject it into all test functions.

Q: What is a REST API?

REST (Representational State Transfer) is an architectural style using HTTP methods, stateless communication, and resource-based URLs.

Q: What tool do you use for API testing?

Postman for manual/exploratory testing. Python + Requests + Pytest for automation. Newman for CI/CD integration.

Q: How to test an API without documentation?

Use browser DevTools Network tab to capture requests. Use Postman to explore endpoints. Read error messages carefully.

Q: What is a test fixture in Pytest?

A reusable setup/teardown function. E.g., auth_token fixture logs in once and provides the token to all tests that need it.

Q: Difference between functional and negative testing?

Functional: Does the API work with valid inputs? Negative: Does it handle invalid inputs gracefully with proper error codes?

Q: What is response time and what's acceptable?

Time from request sent to response received. < 200ms excellent, < 500ms good, < 2s acceptable. > 2s should fail your test.

Q: What is JSON Schema validation?

Verifying that the response JSON has the correct structure — all required fields exist, data types are correct, format is valid.



Quick Reference Card

Save this page — you'll use it every day

HTTP Methods at a Glance

Method	Purpose	Request Body	Success Code
GET	Read/Fetch resource	■	200
POST	Create new resource	■	201
PUT	Replace resource	■	200
PATCH	Partial update	■	200
DELETE	Remove resource	■	204

Must-Assert Checklist

■	Status code matches expected
■	Response time < threshold
■	Required fields present in response
■	Data types correct (str, int, bool)
■	Content-Type: application/json
■	No token → 401, Invalid token → 401
■	Wrong role → 403 (not 401!)
■	Error response has 'message' field

■ Follow for more QA content! Drop this PDF to your LinkedIn — it's your competitive edge in every interview. ■